RAPPORT TECHNIQUE

ShopFlow

Système de Gestion d'une Boutique en Ligne

Projet Backend Spring Boot / Frontend Angular

Mini-Projet — Année Universitaire 2025-2026

Table des Matières

1. Introduction **1**

2. Architecture Technique **1**

2.1 Architecture en couches 1

2.2 Stack technologique 1

3. Modélisation des Données **1**

4. Sécurité et Authentification JWT **1**

5. Choix d'Implémentation Complexes **1**

5.1 Mapping DTO avec MapStruct 1

5.2 Panier persistant et variantes produit 1

6. Difficultés Rencontrées et Solutions **1**

6.1 Implémentation JWT 1

6.2 Intercepteurs HTTP Angular 1

7. Tests Unitaires et Documentation **1**

8. Frontend Angular **1**

9. Conclusion **1**

1. Introduction

ShopFlow est une plateforme e-commerce B2C complète permettant à des vendeurs de publier leurs produits et à des clients d'effectuer des achats en ligne. Développée avec Spring Boot 3 (backend) et Angular (frontend), l'application couvre la chaîne complète d'une boutique numérique : catalogue produits, panier d'achat, commandes avec suivi de statut, et système d'avis clients.

Le système comporte trois profils utilisateurs : ADMIN (gestion de la plateforme), SELLER (gestion de boutique et produits), et CUSTOMER (navigation, panier, commandes). Ce rapport présente l'architecture technique, les choix d'implémentation complexes, et les difficultés rencontrées lors du développement.

2. Architecture Technique

2.1 Architecture en couches

L'application adopte une architecture en couches strictement respectée. Chaque couche communique uniquement avec sa couche adjacente, et les entités JPA ne sont jamais exposées directement : les contrôleurs manipulent exclusivement des DTOs.

Couche	Responsabilité	Annotations clés
Controller	Endpoints REST, validation entrées	@RestController, @Valid
Service	Logique métier, transactions	@Service, @Transactional
Repository	Accès données, JPQL, pagination	JpaRepository, Specification
Entity	Modèle JPA, relations, contraintes	@Entity, relations JPA
DTO	Objets Request/Response séparés	Classes Java pures
Mapper	Conversion Entity ↔ DTO	MapStruct 1.5.5.Final
Exception	Gestion centralisée erreurs	@ControllerAdvice

Le flux de données suit un pattern cohérent : la requête HTTP arrive sur un contrôleur qui valide les données avec @Valid, appelle le service qui orchestre les repositories et applique la logique métier, puis retourne un DTO de réponse via le mapper. Un @ControllerAdvice global intercepte les exceptions et les transforme en réponses HTTP structurées.

2.2 Stack technologique

Technologie	Version	Rôle
Java	21 LTS	Langage backend
Spring Boot	3.2.5	Framework backend
Jakarta EE	Persistence 3, Validation 3	JPA et Bean Validation
Spring Security + JWT	6	Sécurité et authentification
PostgreSQL	16	Base de données relationnelle

Technologie	Version	Rôle
Lombok	—	Réduction boilerplate
Angular	17+	Framework frontend
Springdoc OpenAPI	—	Documentation Swagger UI
JUnit 5 / Mockito	—	Tests unitaires
Maven	—	Build et dépendances

3. Modélisation des Données

Le modèle comporte 12 entités avec des relations JPA complexes. L'entité User centrale définit le rôle via une énumération Role (ADMIN, SELLER, CUSTOMER). Les relations principales sont :

- User — SellerProfile (OneToOne) : Profil boutique avec cascade persistance.
- Product — Category (ManyToMany) : Association multiple via table de jointure JPA.
- Product — ProductVariant (OneToMany) : Variantes avec stock et prix delta individualisés.
- User — Cart (OneToOne) : Panier persistant après déconnexion.
- Cart — CartItem (OneToMany, orphanRemoval) : Lignes supprimées automatiquement.
- Category (auto-référence) : Arbre hiérarchique parent-enfant.

Toutes les relations utilisent FetchType.LAZY par défaut. Les hooks @PrePersist/@PreUpdate gèrent la génération automatique des dates et numéros de commande. Le projet implémente le soft delete : les produits sont marqués actif = false plutôt que supprimés physiquement, préservant l'intégrité des commandes historiques.

La gestion du stock suit une règle stricte : chaque ajout au panier et chaque passage de commande déclenche une vérification en temps réel dans une transaction, avec levée d'exception métier en cas d'insuffisance. Un aspect crucial de la modélisation est que OrderItem.prixUnitaire stocke un snapshot du prix au moment de l'achat, garantissant que les changements futurs de prix produit n'affectent pas les commandes historiques.

4. Sécurité et Authentification JWT

L'authentification repose sur des JWT access tokens signés HMAC-SHA256, implémentés avec Spring Security 6. Un OncePerRequestFilter intercepte chaque requête, extrait et valide le token depuis le header Authorization: Bearer <token>, puis injecte l'authentification dans le SecurityContextHolder. L'endpoint /api/auth/login authentifie les credentials via AuthenticationManager et retourne le token JWT.

Les autorisations sont gérées par @PreAuthorize avec expressions SpEL :

```
@PreAuthorize("hasRole('SELLER') or hasRole('ADMIN')")@PostMappingpublic
ResponseEntity<ProductResponseDTO> createProduct(...) { ...
```

```
}@PreAuthorize("hasRole('ADMIN')")@PutMapping("/status")public
ResponseEntity<OrderResponseDTO> updateOrderStatus(...) { ... }
```

Le SecurityConfig désactive CSRF (API stateless), autorise l'accès public au catalogue, et exige l'authentification pour les autres ressources. Les mots de passe sont hashés avec BCryptPasswordEncoder. Le projet implémente un système complet avec access tokens (durée 15 minutes) et refresh tokens (durée 7 jours), permettant le renouvellement automatique de session via l'endpoint /api/auth/refresh sans redemander les credentials à l'utilisateur.

5. Choix d'Implémentation Complexes

5.1 Mapping DTO avec MapStruct

Le projet utilise MapStruct 1.5.5.Final pour la conversion automatique entre entités et DTOs. Cette bibliothèque génère le code de mapping au moment de la compilation, éliminant le boilerplate manuel tout en conservant des performances optimales .

5.2 Panier persistant et variantes produit

Le panier est stocké en base (OneToOne avec User) plutôt qu'en localStorage. Cette approche garantit la persistance cross-device et permet une synchronisation temps réel du stock. Un panier vide est créé automatiquement à l'inscription via @PostPersist. La collection de CartItem utilise orphanRemoval = true pour un nettoyage automatique.

Les variantes produit (taille, couleur) possèdent chacune un stock supplémentaire et un prix delta optionnel. Le stock total affiché est la somme des stocks des variantes. Lors de l'ajout au panier, le client sélectionne une variante spécifique dont le stock est vérifié individuellement, permettant au vendeur de gérer indépendamment chaque variante.

6. Difficultés Rencontrées et Solutions

6.1 Implémentation JWT

Problème : La plupart des tutoriels en ligne référencent Spring Security 5 avec des configurations obsolètes (WebSecurityConfigurerAdapter, anciennes signatures de filtres), provoquant des incompatibilités difficiles à diagnostiquer avec Spring Boot 3 et Spring Security 6.

Solution : Recours à la documentation officielle Spring Security 6 plutôt qu'aux tutoriels tiers. Migration vers l'approche moderne avec SecurityFilterChain (bean explicite) et configuration fonctionnelle. Réécriture du filtre JWT selon le nouveau modèle de signatures.

6.2 Intercepteurs HTTP Angular

Problème : Difficultés dans la configuration des intercepteurs Angular pour l'injection du token Bearer et la gestion des erreurs 401, avec des subtilités sur la chaîne d'opérateurs RxJS.

Solution : Utilisation d'un HttpInterceptorFn (approche moderne Angular) avec switchMap pour injecter le token depuis un BehaviorSubject, et catchError pour rediriger vers la page de login en cas d'erreur 401. Stockage du token dans localStorage avec une clé application-spécifique.

7. Tests Unitaires et Documentation

La couche Service est couverte par des tests JUnit 5 avec Mockito. Les modules testés incluent AuthService (inscription, login, erreurs), ProductService (CRUD, recherche, soft delete), CartService (ajout, quantités, suppression), et OrderService (passage de commande, statuts, annulation). Les mocks isolent la couche Service des repositories via @ExtendWith(MockitoExtension.class).

La documentation API est générée automatiquement avec Springdoc OpenAPI 3 et accessible sur /swagger-ui. Les annotations @Operation et @ApiResponse enrichissent les descriptions. Une collection Postman complète (shopflow.postman_collection.json) couvre tous les endpoints avec exemples de requêtes et variables d'environnement.

8. Frontend Angular

Le frontend est développé avec Angular 17+ en mode standalone. Le choix d'Angular repose sur son écosystème mature, le typage fort TypeScript, et sa structure rigoureuse facilitant la maintenance.

L'architecture frontend comprend des services par domaine métier encapsulant les appels HTTP, un intercepteur injectant le token JWT et gérant les erreurs, des guards de route protégeant les pages selon les rôles, et des composants réutilisables (ProductCard, RatingStars, OrderStatusBadge). Le design est responsive (mobile-first) avec CSS Grid et Flexbox.

Pages implémentées : accueil avec produits en vedette, catalogue avec filtres dynamiques (catégorie, prix), fiche produit avec variantes et avis, panier avec calcul des totaux, tunnel de commande, espace client (commandes et avis), espace vendeur (dashboard et gestion produits), et authentification login/register.

9. Conclusion

ShopFlow constitue une implémentation complète d'une plateforme e-commerce respectant les spécifications du cahier de charges : architecture en couches, modélisation JPA avancée, sécurité JWT, tests unitaires, et documentation Swagger. Les difficultés principales (JWT et intercepteurs Angular) ont été résolues par une lecture attentive de la documentation officielle.

Améliorations envisageables : système de codes promo avancé, modération collaborative des avis, notifications temps réel de statut via WebSocket, et conteneurisation Docker pour faciliter le déploiement. . Le projet a permis de mettre en pratique l'ensemble des compétences acquises : sécurisation d'applications et construction d'interfaces utilisateur modern.